

Guia de Comandos Git

Fluxo de trabalho para o projeto PsyCool

1. Clonar o repositório

git clone url-do-repositorio – Clona um repositório remoto pra sua máquina

2. Criar / trocar de branch

git branch – Lista as branches locais (a atual vem com *)

git checkout -b nome-da-branch – Cria uma branch nova e já muda pra ela

git checkout nome-da-branch – Troca para uma branch já existente

git fetch – Busca no GitHub as atualizações e novas branches, sem mexer no seu código ainda

git branch -r – Lista somente as branches remotas (que estão no GitHub)

git branch -a – Lista todas as branches, locais e remotas, juntas

git checkout branch_do_colega – Cria (se ainda não existir localmente) e troca pra branch do colega, já rastreando a versão do GitHub

3. Ver o que mudou

git status – Mostra o status dos arquivos, a branch atual, se já foi comitado

git diff – Mostra exatamente o que mudou linha por linha (antes do add)

4. Colocar em staged

git add nome-do-arquivo – Deixa um arquivo específico no staged

git add . – Adiciona todos os arquivos modificados no staged

git rm --cached nome-do-arquivo – Tira o arquivo do staged, volta pro status "modificado"

5. Commitar (salvar localmente)

git commit -m "mensagem descritiva" – Salva as mudanças com uma descrição

6. Ver o histórico (opcional)

git log – Mostra o histórico completo de commits

git log --oneline – Mostra resumido, com o ID de cada commit

7. Subir pro GitHub

git push -u origin nome-da-branch – Envia e "linka" a branch (só na primeira vez)

git push – Envia os commits (nas próximas vezes)

8. Atualizar sua branch com o que os outros fizeram

git fetch – Baixa as atualizações do remoto SEM juntar ainda

git pull – Baixa E já junta as atualizações do remoto na sua branch atual

9. Juntar sua branch com a main

git checkout main – Volta pra branch main

git pull – Garante que a main local está atualizada

git merge nome-da-branch – Junta as mudanças da sua branch na main (traz TUDO que foi commitado nela)

git checkout branch_gui -- nome-do-arquivo – Traz só um arquivo específico de outra branch, sem trazer o resto

10. Comandos de emergência

git checkout -- nome-do-arquivo – Descarta mudanças não commitadas de um arquivo

git stash – Guarda mudanças não commitadas temporariamente (sem commitar)

git stash pop – Traz de volta o que foi guardado no stash

git reset --soft HEAD~1 – Desfaz o último commit, mantendo as mudanças no staged

git reset --hard HEAD~1 – Desfaz o último commit E apaga as mudanças (destrutivo, cuidado)

git remote -v – Mostra qual repositório remoto está conectado

Tipo	Quando usar	Exemplo
feat	Nova funcionalidade	feat: adicionar cadastro de pacientes
fix	Correção de erro	fix: corrigir erro no login
docs	Documentação	docs: atualizar README do projeto
style	Alteração visual/CSS	style: ajustar layout da página inicial
refactor	Melhorar código sem adicionar função	refactor: reorganizar controller de usuários
test	Testes	test: adicionar testes do sistema de login
chore	Configurações ou tarefas gerais	chore: atualizar dependências do projeto

Resumo Rápido

Os comandos que você realmente vai usar no dia a dia

CICLO DO DIA A DIA (decorar esses)

git status — o que mudou?
git add . — manda tudo pro staged
git commit -m "mensagem" — salva localmente
git push — sobe pro GitHub
git pull — baixa o que os outros fizeram

BRANCHES (início de cada funcionalidade nova)

git checkout -b nome-da-branch — cria e troca de branch
git checkout nome-da-branch — troca pra uma que já existe
git branch — quais branches eu tenho aqui

TRABALHANDO COM O TIME (de vez em quando)

git fetch — ver se tem branch nova dos colegas
git branch -a — listar todas as branches (local + remoto)
git checkout branch_do_colega — acessar a branch de alguém
git merge nome-da-branch — juntar uma branch na outra

EMERGÊNCIA (só quando precisar)

git stash / git stash pop — guardar mudanças temporariamente
git checkout -- arquivo — descartar mudança não commitada
git reset --soft HEAD~1 — desfazer último commit (mantém as mudanças)

*Dica: em caso de dúvida na hora de usar **reset --hard** ou **push --force**, pergunte antes de rodar — os dois podem apagar trabalho de forma irreversível.*

Modificado (modified)

Preparado (staged)

Confirmado (committed)